

Capítulo 04

Attention

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 10/08/2026

Capítulo 04 — Attention (o mecanismo de atenção)

Objetivo de aprendizagem: construir a **self-attention** do zero, em quatro versões equivalentes que vão da média mais burra até o mecanismo de verdade. Você vai entender **query/key/value**, a **máscara causal**, o fator de **escala** e o **embedding posicional** — e, no fim, descobrir experimentalmente por que atenção, sozinha, **não basta**.

Pré-requisitos: Capítulos 1-3. Em especial: softmax e loss (Cap. 1), como uma rede aprende (Cap. 2), embeddings e `matmul` em PyTorch (Cap. 3).

Arquivos:

- `attention.py` — o mecanismo em 4 versões, com prova de equivalência
- `model.py` — um modelo de linguagem com atenção, treinado nos nomes
- `exercicios.md` — exercícios

1. O que falta no MLP

O MLP do Capítulo 3 funciona, mas tem duas limitações incômodas:

1. **Ele amassa o contexto.** Os 3 embeddings são **concatenados** num vetor de 30 números e jogados numa camada linear. A posição 1 sempre entra nos mesmos "slots" de peso, a posição 2 nos seguintes, e assim por diante. O modelo não tem como dizer *"neste caso específico, o caractere 1 é o que importa; naquele outro, é o 3"*.
2. **O contexto é caro de aumentar.** Dobrar o contexto dobra o tamanho da primeira camada. Ir para 100 caracteres de contexto seria proibitivo.

O que queremos é diferente: um mecanismo em que cada posição **busca ativamente** a informação de que precisa nas posições anteriores — e essa busca depende do **conteúdo**, não só da posição. Isso é a **atenção**.

A ideia em uma frase: atenção é um mecanismo de **comunicação** — cada posição da sequência olha para as outras e traz de volta uma média **ponderada** delas, onde os pesos são calculados a partir do conteúdo.

Vamos construí-la em quatro versões, cada uma equivalente à anterior. Rode `python attention.py` para acompanhar.

2. Preparando o terreno: a forma dos dados

Trabalhamos com tensores de três dimensões: `(B, T, C)`.

Dimensão	Nome	Significado
B	batch	quantas sequências processamos em paralelo
T	time	quantas posições tem cada sequência
C	channels	quantos números descrevem cada posição

```
B, T, C = 4, 8, 2
x = torch.randn(B, T, C)
```

A tarefa das próximas seções: para cada posição `t`, produzir um vetor que **combine informação das posições 0 até t** — nunca do futuro (isso é essencial, e a Seção 7 explica por quê).

3. v1 — a média na mão (*bag of words*)

A forma mais simples de "combinar o passado" é tirar a **média** dos vetores anteriores. Sem pesos, sem esperteza: todos contam igual.

```
xbow1 = torch.zeros((B, T, C))
for b in range(B):
    for t in range(T):
        xprev = x[b, : t + 1] # tudo até a posição t
        xbow1[b, t] = xprev.mean(dim=0) # média ao longo do tempo
```

Isso se chama *bag of words* ("saco de palavras"): a informação de ordem é perdida, sobra só a mistura. É fraco, mas é o nosso ponto de partida — e, curiosamente, a estrutura desse cálculo já é a da atenção.

4. v2 — a mesma média, com uma multiplicação de matrizes

O loop acima é lento. Aqui entra um truque bonito: **essa média é uma multiplicação de matrizes**.

Monte uma matriz `wei` de $T \times T$ onde a linha `t` tem $1/(t+1)$ nas colunas `0..t` e zero no resto:

```
wei = x = wei @ x =
[1.00 0.00 0.00 0.00] [x0] [x0 ]
[0.50 0.50 0.00 0.00] @ [x1] = [(x0+x1)/2 ]
[0.33 0.33 0.33 0.00] [x2] [(x0+x1+x2)/3 ]
[0.25 0.25 0.25 0.25] [x3] [(x0+...+x3)/4 ]

wei = torch.tril(torch.ones(T, T)) # triangular inferior: zera o futuro
wei = wei / wei.sum(dim=1, keepdim=True) # normaliza cada linha para somar 1
xbow2 = wei @ x # (T,T) @ (B,T,C) -> (B,T,C)
```

Duas peças importantes aparecem aqui:

- **torch.tril** (*triangular lower*) é o que impede olhar para o futuro: as posições acima da diagonal são zero.
- **Cada linha soma 1**, então o resultado é uma média (uma combinação convexa) e não uma soma que explode.

O script confirma que `v1 == v2`:

```
v1 == v2 ? True
(diferença máxima v1 vs v2: 3.24e-08)
```

Sobre esse 3.24e-08: os dois resultados não são *bit a bit* idênticos, e isso é esperado. Números `float32` têm precisão finita, e somar na ordem do loop dá um arredondamento ligeiramente diferente de somar via `matmul`. Por isso comparamos com `torch.allclose(..., atol=1e-6)`: em ponto flutuante, "igual" significa "igual dentro de uma tolerância". Guarde isso — no Capítulo 9 a precisão numérica vira o assunto principal.

5. v3 — a mesma média, via softmax

Agora um passo que parece só uma complicação, mas é **a chave** para a versão final. Em vez de escrever as frações à mão, vamos partir de **zeros**, marcar o futuro com `-infinito` e aplicar **softmax**:

```
tril = torch.tril(torch.ones(T, T))
wei3 = torch.zeros((T, T))
wei3 = wei3.masked_fill(tril == 0, float("-inf")) # proíbe ver o futuro
wei3 = F.softmax(wei3, dim=-1)
xbow3 = wei3 @ x
```

Por que isso dá exatamente a mesma média? Porque o softmax exponencia e normaliza: `exp(0) = 1` nas posições permitidas e `exp(-inf) = 0` no futuro. Uma linha com `k` uns e o resto zeros, normalizada, vira `1/k` em cada posição permitida — a média uniforme. E o script confirma: `v2 == v3 ? True`.

Mas agora olhe o que ganhamos: aqueles zeros iniciais eram um *chute* de que todas as posições importam igual. Se em vez de zeros colocarmos **pontuações calculadas a partir do conteúdo**, o softmax as transforma em pesos que somam 1 — e a média deixa de ser uniforme. É isso que a atenção faz.

6. v4 — self-attention de verdade: query, key e value

Cada posição gera **três** vetores diferentes a partir do seu conteúdo, por três camadas lineares distintas:

Vetor	Pergunta que ele responde
query (q)	"o que eu estou procurando?"
key (k)	"o que eu tenho a oferecer?"
value (v)	"o que eu de fato entrego, se você me escolher"

A afinidade entre a posição **t** e a posição **i** é o **produto escalar** $q[t] \cdot k[i]$: alto quando o que **t** procura casa com o que **i** oferece. Calculamos todas as afinidades de uma vez com uma matmul:

```
k = key(x) # (B, T, head_size)
q = query(x) # (B, T, head_size)
v = value(x) # (B, T, head_size)

wei = q @ k.transpose(-2, -1) # (B, T, T): todas as afinidades
wei = wei * head_size ** -0.5 # escala (Seção 7)
wei = wei.masked_fill(tril == 0, float("-inf")) # máscara causal
wei = F.softmax(wei, dim=-1)

out = wei @ v # a média ponderada dos values
```

Por que `k.transpose(-2, -1)`? Para multiplicar **q** por **k** e obter uma matriz $T \times T$, precisamos que a dimensão de **head_size** seja contraída. `transpose(-2, -1)` troca as duas últimas dimensões de **k**, de (B, T, hs) para (B, hs, T) ; aí a matmul $(B, T, hs) @ (B, hs, T)$ resulta em (B, T, T) . Usamos índices negativos para não mexer na dimensão de batch.

E por que separar **key** de **value**? Porque "como eu sou encontrado" e "o que eu entrego" são coisas diferentes. A **key** serve para o casamento (o endereço); a **value** é o conteúdo que trafega. Separá-las dá liberdade ao modelo.

Rodando, os pesos da primeira sequência ficam assim (linha = quem olha, coluna = quem é olhado):

```
[[1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00]
 [0.52 0.48 0.00 0.00 0.00 0.00 0.00 0.00]
 [0.33 0.30 0.37 0.00 0.00 0.00 0.00 0.00]
 [0.23 0.20 0.27 0.30 0.00 0.00 0.00 0.00]
 ...
 [0.10 0.16 0.08 0.07 0.08 0.25 0.10 0.15]]
```

Note as três propriedades: **triangular** (nada do futuro), **cada linha soma 1**, e — o ponto crucial — os pesos **não são mais uniformes**. Na última linha, a posição 5 recebe **0.25** enquanto a 3 recebe **0.07**: o modelo está *escolhendo* onde olhar.

7. Dois detalhes que fazem o mecanismo funcionar

A escala $1/\sqrt{\text{head_size}}$

Sem ela, o produto escalar $q \cdot k$ cresce com a dimensão (é a soma de `head_size` produtos). Pontuações grandes fazem o softmax **saturar**: a distribuição vira quase um *one-hot*, a atenção "trava" numa única posição e — pior — o gradiente do softmax saturado é quase zero, então o modelo para de aprender (o mesmo efeito da `tanh` saturada que vimos no Cap. 2).

O `attention.py` demonstra na prática:

```
sem escala (valores grandes): tensor([0.80, 0.00, 0.16, 0.03])
com escala (divididos): tensor([0.33, 0.17, 0.27, 0.22])
```

Dividir por $\sqrt{\text{head_size}}$ mantém a variância das pontuações em torno de 1, independentemente do tamanho da cabeça. É por isso que o artigo original se chama "*Attention Is All You Need*" e a fórmula aparece como $\text{softmax}(QK^T/\sqrt{d})V$.

A máscara causal

Estamos treinando um modelo que **prevê o próximo** caractere. Se a posição t pudesse ver a posição $t+1$, ela veria **a própria resposta** — a loss de treino despencaria e o modelo não aprenderia nada útil, porque na hora de gerar texto o futuro não existe. Isso se chama **vazamento** (*data leakage*).

A máscara triangular garante que cada posição só veja o passado e a si mesma. Modelos como o GPT são chamados de **decoder-only causais** exatamente por causa disso. (No exercício E2 você remove a máscara e vê o estrago.)

8. Atenção não sabe ordem: o embedding posicional

Aqui está uma propriedade contra-intuitiva: **a atenção é invariante a permutações**. Ela calcula afinidades entre pares de posições, mas nada no mecanismo diz *qual* vem antes. Se você embaralhasse os tokens de entrada, os pesos acompanhariam o embaralhamento e o resultado seria o mesmo conjunto — `ana` e `naa` seriam indistinguíveis.

Para uma linguagem, isso é fatal: ordem é tudo. A solução é **injetar a posição na entrada**, somando ao embedding do token um embedding que depende de *onde* ele está:

```
self.token_emb = nn.Embedding(vocab_size, n_embd) # o que é o token
self.pos_emb = nn.Embedding(block_size, n_embd) # onde ele está

tok = self.token_emb(idx) # (B, T, n_embd)
pos = self.pos_emb(torch.arange(T)) # (T, n_embd)
x = tok + pos # conteúdo + posição
```

Somar (em vez de concatenar) é o padrão: mantém a dimensão e funciona bem na prática, porque a rede aprende a separar as duas contribuições. Aqui as posições são **aprendidas**; o artigo original usava senos e cossenos fixos, e modelos modernos como o Llama usam **RoPE** (no apêndice do curso). No exercício E3 você remove o `+ pos` e mede o prejuízo.

9. O resultado — e a descoberta honesta deste capítulo

O `model.py` treina um modelo de linguagem com **uma cabeça de atenção**, contexto de **8** caracteres (contra 3 do MLP) e ~11,4k parâmetros — praticamente os mesmos ~11,9k do MLP do Capítulo 3, para a comparação ser justa.

Rode e compare:

Modelo	Contexto	Parâmetros	Loss validação
Bigrama (Cap. 1)	1	729	~2,4
MLP (Cap. 3)	3	11.897	1,967
Atenção	3	11.103	2,193
Atenção	8	11.363	2,099
Atenção + feedforward	8	33.255	1,913

Leia essa tabela com atenção, porque ela contradiz o que você provavelmente esperava:

1. Mais contexto ajuda a atenção. Passando de 3 para 8 caracteres, a loss cai de 2,193 para 2,099. O mecanismo consegue usar o contexto extra — e, diferente do MLP, sem explodir o número de parâmetros.

2. Mas a atenção sozinha *perde* do MLP (2,099 contra 1,967), mesmo com o mesmo orçamento de parâmetros. Isso **não** é bug. É a lição central do capítulo.

Por quê? Olhe o que a atenção realmente faz: `out = wei @ v`. Dados os pesos, isso é uma **média ponderada** dos values — uma operação essencialmente **linear**. A atenção é excelente em **comunicar** (mover informação entre posições), mas ela não **processa** essa informação: falta o "pensar" não-linear em cada posição, que é justamente o que o MLP do Capítulo 3 fazia bem com a sua camada oculta e a GELU.

3. A prova: ligue `USE_FEEDFORWARD = True` no `model.py`. Isso adiciona apenas uma pequena rede por posição (expande, GELU, volta) depois da atenção:

```
self.net = nn.Sequential(
    nn.Linear(dim, 4 * dim),
    nn.GELU(),
    nn.Linear(4 * dim, dim),
)
```

A loss cai para **1,913** — melhor que o MLP. Note que o modelo fica maior (33k parâmetros), então não é uma comparação de orçamento igual; o ponto é **direcional** e é o recado do capítulo:

Atenção = comunicação. Feedforward = computação. O Transformer é a combinação dos dois, empilhada em camadas. Nenhum dos dois sozinho é suficiente.

Esse é exatamente o desenho do bloco do Transformer, e é o Capítulo 5.

Os nomes gerados, para constar, já saem bem plausíveis (do modelo só com atenção):

```
jasoni, jovaldimar, spatie, cemindray, luedie, josmar, deseni, alcindilo, ...
```

Repare que a loss e a "beleza" dos nomes não andam perfeitamente juntas — a loss é a métrica objetiva e é nela que confiamos para comparar modelos.

10. Resumo do capítulo

- A atenção resolve as limitações do MLP: o modelo **escolhe** onde olhar, com base no **conteúdo**, e o contexto cresce sem explodir os parâmetros.
- Construímos o mecanismo em 4 versões equivalentes: média num loop -> matmul com **tril** -> softmax com máscara **-inf** -> **query/key/value**.
- **$\text{softmax}(\mathbf{QKT}/\sqrt{d})\mathbf{V}$** : afinidades por produto escalar, escaladas por $1/\sqrt{d}$ (evita saturação do softmax), mascaradas (proíbe ver o futuro), normalizadas, e usadas para ponderar os **values**.
- A atenção é **invariante a permutações** -> precisa de **embedding posicional**.
- Comparação com precisão de ponto flutuante exige **tolerância** (**atol**), porque **float32** não é exato.
- **A descoberta honesta**: atenção sozinha não bate o MLP. Ela **comunica** mas não **computa**. Atenção + feedforward bate. Isso motiva o Transformer.

O que vem no Capítulo 5

Temos as duas metades: comunicação (atenção) e computação (feedforward). O **Capítulo 05 — Transformer** junta as duas num **bloco**, adiciona **múltiplas cabeças** (várias relações em paralelo), **conexões residuais** e **layer normalization** — e empilha tudo em profundidade. O resultado é a arquitetura do GPT-2.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 04 (Attention)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

E1 — Leitura de código (aquecimento)

Sem rodar, responda:

1. O que a matriz `tril` (triangular inferior) impede? O que aconteceria com o **treino** se o modelo pudesse ver o futuro?
 2. Por que dividimos as pontuações por `sqrt(head_size)`?
 3. Qual a diferença de papel entre **query**, **key** e **value**?
-

E2 — Sem a máscara causal (vazamento de dados)

No `model.py`, comente a linha do `masked_fill` (a máscara causal) e treine de novo.

1. A loss de treino cai mais rápido? Ela fica **artificialmente** boa?
 2. Gere nomes com esse modelo. A qualidade acompanha a loss?
 3. Explique por que remover a máscara é **trapaça**: o modelo passa a usar a resposta para prever a resposta.
-

E3 — Sem o embedding posicional

Remova o `+ pos` do `forward` (use apenas `tok`) e treine de novo.

1. A loss piora? Quanto?
 2. **Por quê?** Sem posição, a atenção vê o contexto como um *conjunto* e não como uma *sequência* — ou seja, `ana` e `naa` ficariam indistinguíveis. Explique com suas palavras por que isso atrapalha na hora de prever o próximo caractere.
-

E4 — Sem a escala `sqrt(head_size)`

Remova o fator `* k.shape[-1]**-0.5`.

1. O que acontece com a loss no começo do treino?
 2. Imprima os pesos de atenção (`wei`) de um batch. Eles ficam mais "concentrados" (perto de 0 e 1) do que com a escala?
 3. Relacione com o experimento do fim do `attention.py`: softmax de valores grandes satura, e gradiente saturado quase não aprende.
-

E5 — Tamanho do contexto (`block_size`)

Treine com `block_size` = 3, 8 e 16.

1. Anote a loss de validação. Mais contexto sempre ajuda?
2. Para nomes (palavras curtas, ~7 letras), faz sentido um contexto de 16? O que o embedding posicional aprende nas posições que quase nunca são usadas?
3. Como o custo de computação da atenção cresce com o `block_size`? (Dica: a matriz de afinidades é $T \times T$.)

E6 — Inspeccionando o que o modelo olha (desafio)

Faça o modelo devolver também os pesos `wei` e, para um nome específico (ex.: o contexto `. . . . .` `a n a`), imprima a linha da última posição.

1. Em quais posições o modelo mais presta atenção ao prever o próximo caractere?
2. Você provavelmente vai achar **muito peso nos tokens de preenchimento** `.`, e não nas letras. Antes de chamar isso de bug, pense: o softmax **obriga** os pesos a somarem 1. Se o modelo não precisa trazer informação de nenhuma posição específica, onde ele "estaciona" esse peso? (Esse fenômeno tem nome em LLMs grandes: *attention sink*.)

A distribuição é claramente **não uniforme** — compare com o *bag of words* da Seção 3, onde todos os pesos eram iguais. O que isso mostra sobre o mecanismo?

Solução de referência: `solucoes/e6_inspeccionar_atencao.py`.

E7 — Multi-head attention (desafio, antecipa o Cap. 5)

Uma cabeça só aprende **um** tipo de relação. Implemente **multi-head attention**: crie `n_heads` cabeças com `head_size = n_embd // n_heads`, rode todas em paralelo e **concatene** as saídas.

1. Com o mesmo total de parâmetros, 4 cabeças pequenas vão melhor que 1 grande?
2. Por que ter várias cabeças pode ajudar? (Pense: uma cabeça olha a vogal anterior, outra o começo da palavra...)
3. Essa é exatamente a peça que abre o Capítulo 5 — o Transformer.